

Hybrid Sparse+Dense Retrieval: Implementation and Comparative Study

Hema Varshini
mallarapuv@iitbhillai.ac.in
Indian Institute of Technology Bhilai
Bhilai, Chhattisgarh, India

Shubham Mahajan
shubhamy@iitbhillai.ac.in
Indian Institute of Technology Bhilai
Bhilai, Chhattisgarh, India

Vedansh Sharma
vedanshs@iitbhillai.ac.in
Indian Institute of Technology Bhilai
Bhilai, Chhattisgarh, India

Abstract

Hybrid retrieval systems that combine sparse lexical signals (e.g., BM25) with dense semantic embeddings have recently emerged as an effective option other than to do purely sparse or purely dense retrieval. However, reproducing and evaluating hybrid fusion methods still remains challenging due to incompatible score scales and the lack of open-source hybrid implementations.

In this work, we implement a complete sparse+dense hybrid retrieval pipeline, and also evaluate it across 23 BEIR datasets, we compare 3 normalization strategies (min-max, z-score, rank) under fixed and heuristic fusion weights. Our experiments show that hybrid retrieval consistently improves ranking metrics such as nDCG@10, MRR@10, Precision@10, Recall@10, and MAP@100 over BM25 and dense MiniLM baselines. We also demonstrate that a globally fixed mixing weight such as $\alpha = 0.4$ provides strong performance across datasets. This study provides a reproducible framework for hybrid retrieval analysis under realistic compute constraints.

Keywords

Information Retrieval, Hybrid Retrieval, Sparse Retrieval, Dense Retrieval, BEIR, Score Normalization

ACM Reference Format:

Hema Varshini, Shubham Mahajan, and Vedansh Sharma. 2025. Hybrid Sparse+Dense Retrieval: Implementation and Comparative Study. In *Proceedings of the ACM Conference (YourConf '25)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/XXXXXXX.XXXXXXX>

1 Introduction

Sparse retrieval methods such as BM25 remain widely used owing to their interpretability, scalability, and strong performance on keyword heavy queries. Dense retrieval powered by transformer encoders such as DPR and MiniLM excels at semantic matching even under vocabulary mismatch. Both families exhibit complementary strengths: sparse models reward exact lexical overlap, while dense retrievers capture latent semantic similarity.

Hybrid retrieval seeks to combine these strengths into unified scoring model. Score-level fusion, in particular, enables light weight

integration of heterogeneous signals. However, hybrid retrieval is sensitive to scaling, normalization and the choice of mixing weight. Reproducing published hybrid systems is further complicated by the lack of publically available code.

The objective of this project is to:

- implement a full sparse+dense retrieval pipeline from scratch
- perform score normalization and linear fusion
- evaluate fusion across 23 BEIR datasets
- study the effect of multiple normalisations and mixing weights.

This work is intended as a practical and reproducible study rather than a new neural architecture.

2 Related Work

Sparse retrieval is dominated by classical models such as BM25 and query likelihood. Dense retrieval, introduced with neural encoders such as DPR, ColBERT, MiniLM, and MPNet, provides strong semantic matching.

A recent research direction is hybrid retrieval. Zhang et al. (2024) analyze hybrid sparse-dense vector search graphs and show that fusion improves both accuracy and efficiency. Other hybrid approaches include weighted interpolation, rank-based fusion and neural hybrid scoring models. Our work focuses on linear fusion with normalisation, which is the most accessible and reproducible variant.

3 Datasets

We evaluate our approach on 23 BEIR datasets spanning multiple domains:

- biomedical: trec-covid, scifact
- fact checking: scifact
- scientific retrieval: scidocs
- financial QA: fiqa
- passage retrieval: nfcampus
- community QA: 12 datasets from cqadupstack

All datasets are accessed using `ir_datasets` python API.

4 Linear

4.1 Sparse Retrieval: BM25

We use BM25 scores provided through `ir_datasets`. Each run is stored in TREC format as:

$$\text{BM25}(q, d).$$

4.2 Dense Retrieval: MiniLM + FAISS

We use the SentenceTransformer `all-MiniLM-L6-v2` encoder to compute both query and document embeddings. FAISS IndexFlatIP

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

YourConf '25, Bhilai, India

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/25/02
<https://doi.org/10.1145/XXXXXXX.XXXXXXX>

is used for nearest neighbor search. Dense scores correspond to cosine similarity:

$$\text{Dense}(q, d) = \langle \mathbf{q}, \mathbf{d} \rangle.$$

Top-100 candidates are retrieved.

4.3 Score Normalization

Sparse and dense scores differ in scale. We evaluate three normalization strategies:

(1) **Min-Max Normalization**

$$s' = \frac{s - \min}{\max - \min}$$

(2) **Z-score Normalization**

$$s' = \frac{s - \mu}{\sigma}$$

(3) **Rank Normalization**

$$s' = \frac{N - \text{rank}(d)}{N - 1}$$

4.4 Hybrid Fusion

The hybrid score integrates normalised sparse and dense scores:

$$S_{\text{hybrid}}(q, d) = \alpha S_{\text{sparse}}(q, d) + (1 - \alpha) S_{\text{dense}}(q, d).$$

We experiment with:

$$\alpha \in \{0.4, 0.5, 0.6\}$$

and a heuristic value α_h computed from dataset metadata (document count, query count, average relevance density).

```
{
  "docs": {
    "count": 171332,
    "fields": {
      "doc_id": {
        "max_len": 8,
        "common_prefix": ""
      }
    }
  },
  "queries": {
    "count": 50
  },
  "qrels": {
    "count": 66336,
    "fields": {
      "relevance": {
        "counts_by_value": {
          "2": 14217,
          "1": 10456,
          "0": 41661,
          "-1": 2
        }
      }
    }
  }
}
```

Figure 1: Example metadata of the beir/trec-covid dataset.

4.5 Heuristic α Computation.

Given dataset metadata consisting of:

N_d = number of documents, N_q = number of queries, N_r = number of relevant documents

we define the average number of relevant documents per query as:

$$\bar{r} = \frac{N_r}{\max(1, N_q)}.$$

Two dataset-dependent scaling factors are computed:

$$\text{doc_factor} = \text{clip}\left(\frac{\log_{10}(N_d)}{6}, 0, 1\right),$$

$$\text{query_factor} = \text{clip}\left(\frac{N_q}{10000}, 0, 1\right).$$

We then construct two bias terms:

$$\alpha_{\text{BM25}} = 1 - 0.5 \text{ query_factor} - 0.3 \text{ doc_factor},$$

$$\alpha_{\text{dense}} = 1 - \text{clip}\left(\frac{\bar{r}}{100}, 0, 1\right).$$

The final heuristic mixing wt is obtained by averaging both bias and clipping to the interval $[0.2, 0.8]$:

$$\alpha_h = \text{clip}(0.5 \alpha_{\text{BM25}} + 0.5 \alpha_{\text{dense}}, 0.2, 0.8).$$

For implementation purpose, α_h is rounded to 2 decimals.

5 Experimental Setup

5.1 Evaluation Metrics

Following BEIR conventions and the hybrid retrieval paper cited in our SOP, we compute:

- nDCG@10,
- MRR@10,
- Precision@10,
- Recall@10,
- MAP@100.

5.2 Baselines Compared

We compare:

- BM25 (sparse),
- MiniLM dense,
- Hybrid with fixed α ,
- Hybrid with heuristic α .

All retrieval runs use $\text{top_k} = 100$ candidates.

6 Results

6.1 Fixed vs Heuristic α

We compare three fixed values $\alpha \in \{0.4, 0.5, 0.6\}$ and a dataset-dependent heuristic α_h . To determine which α should be used in the hybrid retriever to compare with dense and sparse retrievers

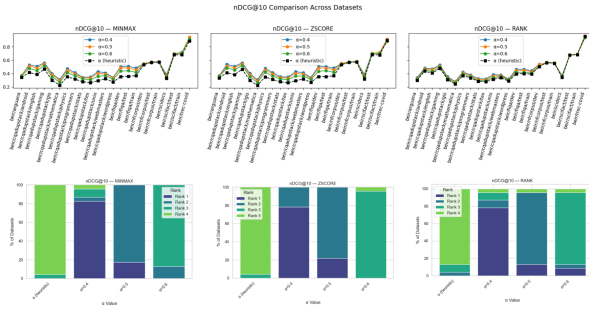


Figure 2: nDCG@10 comparison across datasets

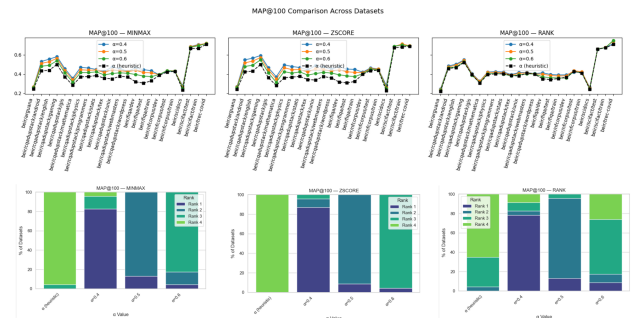


Figure 6: MAP@100 comparison across datasets

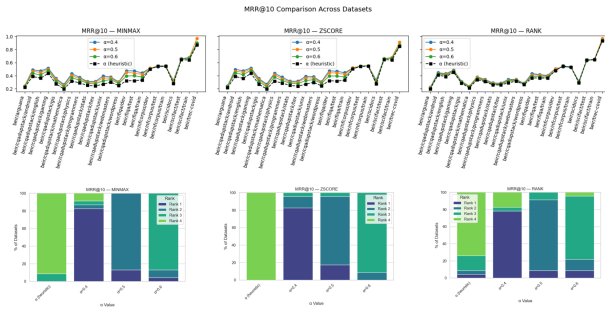


Figure 3: MRR@10 comparison across datasets

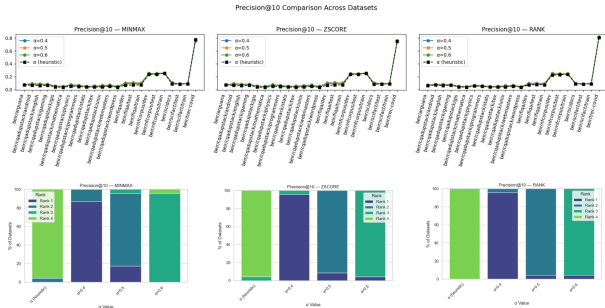


Figure 4: Precision@10 comparison across datasets

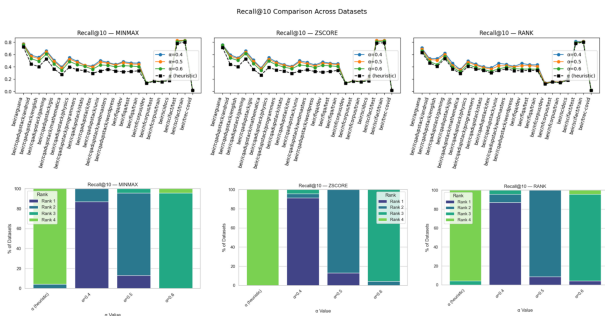


Figure 5: Recall@10 comparison across datasets

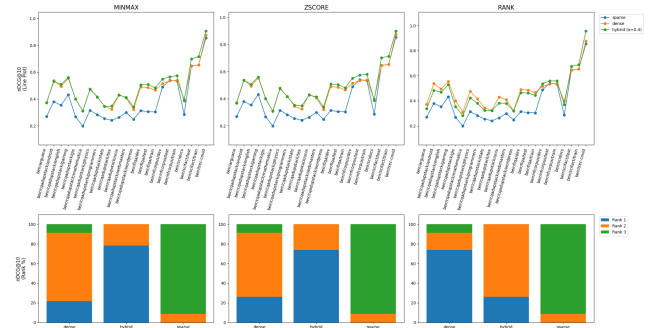


Figure 7: nDCG@10 comparison across datasets

Across all 23 BEIR datasets, we consistently observed that a fixed fusion weight of $\alpha = 0.4$ delivered the strongest performance among all tested values. This trend is held uniformly across all 2 normalisation strategies: min-max, z-score, and rank—indicating that $\alpha = 0.4$ is a generally robust mixing coefficient for hybrid sparse+dense retrieval. Intuitively, this setting assigns slightly more weight to the dense semantic similarity signal while still preserving a substantial contribution from BM25’s lexical precision. Most BEIR datasets contain a mixture of para phrastic queries and keyword sensitive cases; as a result dense retrieval often captures the broader semantic context while BM25 anchors the ranking with exact term matches. The balance achieved at $\alpha = 0.4$ appears to align well with this heterogeneous signal distribution thus enabling the hybrid model to outperform both BM25 and MiniLM on the majority of datasets.

6.2 Sparse vs Dense vs Hybrid

Hybrid retrieval (with $\alpha = 0.4$ in Hybrid Score computation) outperforms both BM25 and dense retrieval across most datasets and metrics.

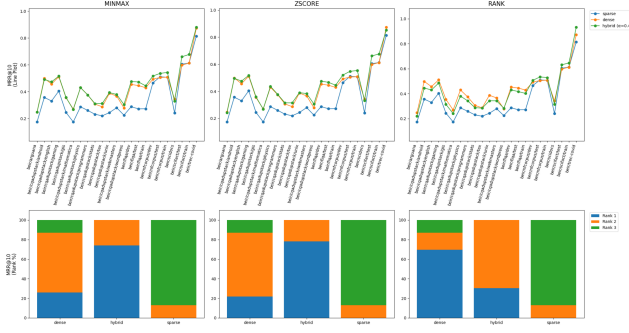


Figure 8: MRR@10 comparison accross datasets

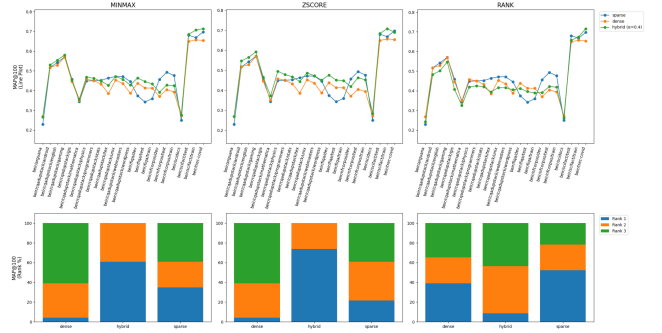


Figure 11: MAP@100 comparison accross datasets

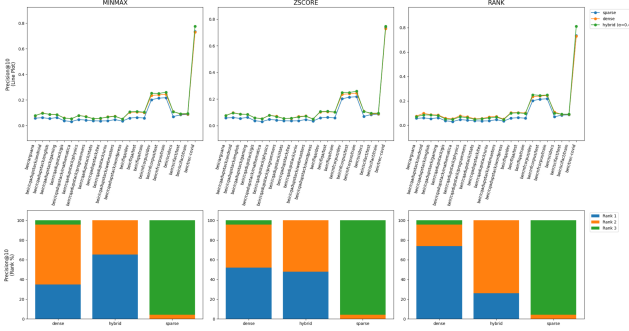


Figure 9: Precision@10 comparison accross datasets

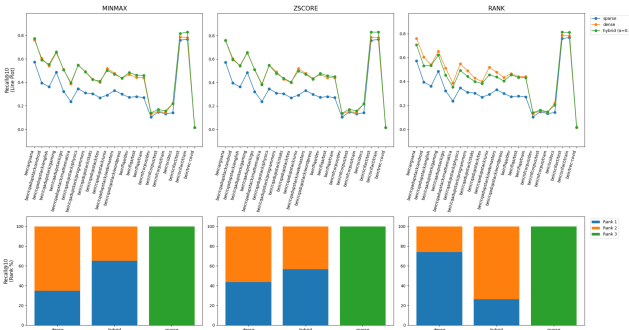


Figure 10: Recall@10 comparison accross datasets

Across all BEIR datasets and evaluation metrics, the hybrid retriever with a fixed fusion weight of $\alpha = 0.4$ demonstrates strong and consistent performance. Under both min-max and z-score normalisation the hybrid model achieved Rank 1 on more than 50% of the datasets for every metric hence indicating that the linear combination of sparse and dense signals provides a superior retrieval signal. In the remaining cases, the hybrid model is almost always ranked second, typically surpassed only by dense retrieval. Quite importantly, hybrid retrieval outperforms the sparse BM25 baseline on more than 90% of the datasets, confirming that incorporating semantic similarity significantly strengthens retrieval robustness.

A notable deviation occurs under rank normalisation: in this setting, dense retrieval overtakes the hybrid model in more than 50% of the datasets, pushing hybrid to Rank-2. This behavior can be explained by the nature of rank normalization, which compresses the score differences between sparse and dense systems and reduces the influence of absolute score values. Because MiniLM embeddings generally provide stronger semantic ordering than BM25, rank-based normalisations tend to amplify the relative consistency of dense similarity scores while reducing the lexical advantage of the sparse component. Despite this shift, the hybrid system still outperforms the sparse model in more than 90% of the cases, highlighting that some contribution from semantic matching remains beneficial even when rank normalization limits the expressiveness of the hybrid score.

Overall these findings indicate that $\alpha = 0.4$ achieves a strong balance between lexical precision and semantic generalization for most normalisation strategies. The hybrid model benefits from the complementary strengths of BM25 and MiniLM, leading to superior performance in min-max and Z-score settings. Although rank normalization favors dense retrieval, hybrid retrieval remains consistently superior to purely sparse retrieval, confirming the robustness of score-level fusion across diverse datasets.

7 Vector Generation Procedure

This section describes the preprocessing pipeline used to construct sparse BM25 vectors and dense semantic embeddings for all document and queries in the data set. These representation form the foundation for sparse, dense, and hybrid retrieval experiments.

7.1 Loading Corpus and Queries

We begin by loading the input files `corpus.jsonl` and `queries.jsonl`. From these files, the system extracts:

- document identifiers and document txt
- query identifiers and query text.

These form the raw inputs for both sparse and dense vector generation.

7.2 BM25 Sparse Document Vectors

Vocabulary Construction. A global vocabulary is constructed by tokenising all documents. For each token, we compute:

- document frequency(DF)
- average document length across the corpus

BM25 Weighting. Using the computed statistics, standard BM25 inverse document frequency(IDF) values are obtained. Each document is then converted into a sparse BM25 vector in which each nonzero component corresponds to the BM25 weight of a term appearing in that document.

Storage Formats. Document BM25 vectors are saved in two parallel formats:

- a single compressed sparse row (CSR) matrix stored as `bm25_docs_csr.npy`
- individual per-document JSONL entries (Pinecone-style) for compatibility with vector stores.

7.3 BM25 Sparse Query Vectors

Queries are encoded using the *same* BM25 vocabulary, DF values, IDF weights, and length statistics as the documents, ensuring compatibility during retrieval. Out-of-vocabulary(OOV) tokens are ignored, since no new vocabulary is introduced at query time.

The resulting sparse query vectors are stored in `bm25_queries.jsonl`.

7.4 Dense Vector Generation

A Sentence-Transformers encoder (MiniLM-L6-v2 by default) is used to map:

- each document
- each query

into dense semantic embeddings. Optional L2 normalization allows cosine similarity and inner-product retrieval to behave consistently.

Dense vectors are saved both as:

- JSONL files for readability and metadata
- .npy arrays for fast loading in downstream retrieval tasks.

7.5 Final Output

The vector generation stage produces two sets of representations:

- **Sparse BM25 vectors** for documents and queries (CSR matrix + JSONL).
- **Dense transformer embeddings** for documents and queries (.jsonl + .npy).

Together, these representations form the complete feature set used for:

- sparse retrieval (BM25),
- dense retrieval (transformer embeddings),

- hybrid retrieval experiments (e.g., SUM fusion, Hadamard fusion, RRF, CombSUM, cross-attention fusion).

8 Hybrid Hadamard Fusion: Dense

$\odot(1 + \lambda_s \cdot \text{Projected Sparse})$

Beyond simple additive fusion, we explore a multiplicative (Hadamard) fusion scheme that allows sparse BM25 evidence to gate or reweight individual dimensions of the dense embedding. The purpose is not merely to add lexical and semantic signals, but to let the sparse signal shape the dense representation in a feature wise manner.

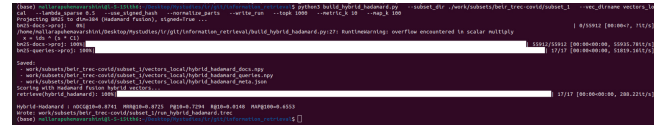


Figure 12: Code

8.1 Hashing-Based Projection of BM25 (Same as SUM Method)

As in SUM-fusion, the high-dimensional sparse BM25 vector is projected to fixed dimensional dense vector using the hashing trick. Each term with BM25 weight is mapped one of D buckets via:

$$\text{bucket}(t) = \text{term_id}(t) \bmod D.$$

A deterministic signed hash assigns each term a sign $\in \{+1, -1\}$ so collision may cancel or amplify:

$$\phi_{\text{BM25}}(x) \in \mathbb{R}^D.$$

Optionally, both the dense embeddings and ϕ_{BM25} may be L2-normalized to stabilize fusion.

Advantages.

- Dimension is fixed and independent of vocabulary size.
- Efficient, backend-friendly representation of sparse signals.
- Signed hashing reduces collision bias.

8.2 Multiplicative Gating via Hadamard Fusion

Instead of adding dense and projected sparse vectors the BM25 projection is used as a dimension-wise gate over the dense embedding.

For a document(or a query), the gate vector is:

$$\text{gate}(x) = 1 + \lambda_s \cdot \tanh(\phi_{\text{BM25}}(x)),$$

where:

- $\phi_{\text{BM25}}(x)$ is the hashed BM25 projection,
- $\tanh(\cdot)$ squashes values to $(-1, 1)$,
- λ_s controls sparse modulation strength.

The hybrid embedding becomes:

$$h(x) = d_{\text{dense}}(x) \odot \text{gate}(x),$$

i.e.,

$$h(x) = d_{\text{dense}}(x) \odot (1 + \lambda_s \cdot \tanh(\phi_{\text{BM25}}(x))),$$

with $\odot$ denoting element-wise Hadamard product.

Intuition.

- Strong BM25 evidence increases weights of certain dense dimensions.
- Signed hashing + tanh permits slight down-weighting when evidence is negative.
- Dense encoder provides semantic structure; BM25 gates emphasize lexical relevance.

Because the same transformation applies to documents and queries, both lie in a shared hybrid embedding space whi is suitable for dot-product retrieval.

8.3 Why Multiplicative (Hadamard) Fusion?

Compared to additive SUM fusion:

Feature-wise modulation. SUM fusion mixes vectors directly, Hadamard fusion keeps the dense geometry intact while amplifying or dampening specific semantic dimensions.

Preserves dense geometry. The dense representation is scaled, NOT replaced, hence preserving semantic angular structure.

Interpretable hyperparameter.

$$\lambda_s = 0 \Rightarrow h(x) = d_{\text{dense}}(x),$$

$$\lambda_s > 0 \Rightarrow \text{progressively stronger lexical gating.}$$

Single dense vector. The output is still a single D -dimensional vector per document/query—compatible with all dense retrieval systems.

8.4 Retrieval and Evaluation

Hybrid Hadamard vectors are used for retrieval via dot product:

$$\text{score}(q, d) = h(q)^\top h(d).$$

Ranking is performed using this score. Evaluation metrics include:

- nDCG@10,
- MRR@10.

Run files are exported in TREC format for external evaluation.

9 Hybrid SUM Fusion with Hashed Sparse Projections

To construct a single unified representation that integrate dense semantic embeddings with sparse BM25 signal, we implement a projection based hybrid SUM fusion. The main idea is project the high-dimensional sparse BM25 vector in the same dimensional space as the dense embedding and combine the 2 via a weighted sum. This yields a single hybrid vector per document or query that can be used with any standard dense retrieval backend.

```

$ python3 hash_bm25.py --dense_dir ./data/dense --bm25_dir ./data/bm25 --output_dir ./output
$ python3 hash_bm25.py --dense_dir ./data/dense --bm25_dir ./data/bm25 --output_dir ./output --vocab ./vocab.txt
$ python3 hash_bm25.py --dense_dir ./data/dense --bm25_dir ./data/bm25 --output_dir ./output --vocab ./vocab.txt --seed 42
$ python3 hash_bm25.py --dense_dir ./data/dense --bm25_dir ./data/bm25 --output_dir ./output --vocab ./vocab.txt --seed 42 --device cuda
$ python3 hash_bm25.py --dense_dir ./data/dense --bm25_dir ./data/bm25 --output_dir ./output --vocab ./vocab.txt --seed 42 --device cuda --workers 4
$ python3 hash_bm25.py --dense_dir ./data/dense --bm25_dir ./data/bm25 --output_dir ./output --vocab ./vocab.txt --seed 42 --device cuda --workers 4 --batch_size 1024
$ python3 hash_bm25.py --dense_dir ./data/dense --bm25_dir ./data/bm25 --output_dir ./output --vocab ./vocab.txt --seed 42 --device cuda --workers 4 --batch_size 1024 --log_dir ./log
$ python3 hash_bm25.py --dense_dir ./data/dense --bm25_dir ./data/bm25 --output_dir ./output --vocab ./vocab.txt --seed 42 --device cuda --workers 4 --batch_size 1024 --log_dir ./log --verbose
$ python3 hash_bm25.py --dense_dir ./data/dense --bm25_dir ./data/bm25 --output_dir ./output --vocab ./vocab.txt --seed 42 --device cuda --workers 4 --batch_size 1024 --log_dir ./log --verbose --help

```

Figure 13: Code

9.1 Hashing-Based Projection of Sparse BM25 Vectors

BM25 representations are inherently high dimensional and sparse. Direct concatenation with dense vectors would produce extremely large representations and make indexing computationally expensive. Instead, we apply the hashing trick to project the BM25 vector into a dense vector of fixed dimension D , chosen to match the dense encoder dimension.

Each BM25 term t with BM25 weight w_t is mapped using:

$$\text{bucket}(t) = \text{term_id}(t) \bmod D,$$

and the wt is accumulated in that bucket. Optionally, a deterministic signed hash assigns each term a sign $s_t \in \{-1, +1\}$ enabling collisions to cancel or reinforce:

$$\phi_{\text{BM25}}(x) \in \mathbb{R}^D.$$

Advantages.

- Fixed dimensionality independent of vocabulary size.
- Memory-efficient and avoids storing large sparse vectors.
- Backend-friendly: projected sparse vectors match the dense embedding dimensions.
- Signed hashing reduces systematic collision bias.

9.2 Part-wise Normalisation (Optional)

Before fusion, we optionally apply L2 normalization to each component:

$$\hat{d}_{\text{dense}} = \frac{d_{\text{dense}}}{\|d_{\text{dense}}\|_2}, \quad \hat{\phi}_{\text{BM25}}(x) = \frac{\phi_{\text{BM25}}(x)}{\|\phi_{\text{BM25}}(x)\|_2}.$$

Why normalization helps.

- Controls scale so that one component does not dominate.
- Makes fusion weights more interpretable.
- Stabilizes training and hyperparameter tuning.

9.3 SUM Fusion: Dense+ Projected Sparse

After projection (and optional normalization), hybrid vectors for documents and queries are formed via a weighted sum:

$$h_{\text{doc}} = \lambda_{\text{dense}} d_{\text{dense}} + \lambda_{\text{sparse}} \phi_{\text{BM25}}(\text{doc}),$$

$$h_{\text{query}} = \lambda_{\text{dense}} q_{\text{dense}} + \lambda_{\text{sparse}} \phi_{\text{BM25}}(\text{query}),$$

where:

- $d_{\text{dense}}, q_{\text{dense}} \in \mathbb{R}^D$ are dense encoder embeddings,
- $\phi_{\text{BM25}}(\cdot)$ are hashed BM25 projections,
- $\lambda_{\text{dense}}, \lambda_{\text{sparse}}$ are scalar fusion weights controlling the contributions of semantic and lexical signals.

The resulting hybrid vectors

$$h_{\text{doc}}, \quad h_{\text{query}} \in \mathbb{R}^D,$$

combine BM25 lexical matching with dense semantic similarity, and fit into standard dense vector search systems.

Advantages.

- Single-vector representation enables standard dense retrieval(eg FAISS).
- Fusion weights allow flexible tuning between semantic and lexical emphasis.
- Computationally simple: fusion is just vector addition.
- Hybrid vectors can be precomputed and reused across experiments.

9.4 Retrieval and Evaluation with Hybrid Vectors

Similarity b/w query and document is computed via the dot product:

$$\text{score}(q, d) = h_{\text{query}}(q)^T h_{\text{doc}}(d).$$

Documents are ranked using this score, and we compute standard IR metrics:

- nDCG@10 (quality at high ranks),
- MRR@10 (how early the first relevant document appears).

A TREC-formatted run file can be generated for use with external evaluation tools.

Conceptual takeaway. This approach turns hybrid sparse+dense retrieval into a pure vectorspace problem, where each item is represented by a compact embedding that smoothly integrates BM25 and dense semantic information through hashing and weighted summation.

10 Hybrid Fusion without Fixed α : Methods Overview

After generating BM25 and dense scores, we evaluate several hybrid score level fusion strategies that do not rely on a manually chosen interpolation parameter α . Instead, we compare classical unsupervised fusion rules and a simple learned linear combination. For all methods, the starting point is:

- a BM25 score for each (q, d) pair,
- a dense encoder score (dot product between dense query and document vectors).

Each fusion method produces a single combined score per document, which is then used to rank documents for that query.

Baseline performance is measured for:

- BM25 only,
- Dense only,

and is compared against the hybrid strategies described below.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# This script demonstrates a simple hybrid retrieval system.
# It takes a query and a list of documents, and returns the top-k results.
# The documents are represented as a list of (id, vector) pairs.
# The query is represented as a vector.
# The script uses a simple dot product to calculate the similarity between the query and the documents.
# The results are sorted by similarity and the top-k are returned.

import sys
import numpy as np

def main():
    # Parse command line arguments
    query = sys.argv[1]
    docs = sys.argv[2:]

    # Convert query and documents to vectors
    query_vec = np.array([float(x) for x in query.split(' ')])
    docs_vecs = []
    for doc in docs:
        id, vec = doc.split(' ')
        docs_vecs.append((id, np.array([float(x) for x in vec.split(' ')])))

    # Calculate dot products
    scores = []
    for (id, vec) in docs_vecs:
        score = np.dot(query_vec, vec)
        scores.append((id, score))

    # Sort by score
    scores.sort(key=lambda x: x[1], reverse=True)

    # Print top-k results
    k = 10
    for i in range(k):
        id, score = scores[i]
        print(id, score)
```

Figure 14: Code

10.1 Evaluation Protocol

For each query, we generate a ranked list of documents and evaluate using:

- **nDCG@k**(normalized Discounted Cumulative Gain)-ranking quality with emphasis on top-ranked positions,
- **MRR@k**(Mean Reciprocal Rank)-measures the position of the first relevant document.

All methods are evaluated on the same query set, allowing direct comparison among sparse, dense, and hybrid approaches.

10.2 Reciprocal Rank Fusion (RRF)

Idea. Reciprocal Rank Fusion is a rank-based fusion method that does not combine raw scores but the ranks produced by each system. The RRF score is:

$$\text{RRF}(d) = \frac{1}{k + \text{rank}_{\text{dense}}(d)} + \frac{1}{k + \text{rank}_{\text{BM25}}(d)},$$

where k is a constant (typically $k = 60$) that dampens the influence of deep-ranked documents.

Advantages.

- **Scale-robust:** depends only on ranking, not score scale.
- **Robust ensemble:** rewards documents ranked reasonably well by both systems.
- **Parameter-light:** only one intuitive hyperparameter k .

10.3 CombSUM with z-score Normalization (CombSUM-z)

Idea. CombSUM is a score-based fusion method. To make BM25 and dense scores comparable, we apply per-query z-normalization:

$$s'_{\text{dense}}(d) = \frac{s_{\text{dense}}(d) - \mu_{\text{dense}}}{\sigma_{\text{dense}}}, \quad s'_{\text{BM25}}(d) = \frac{s_{\text{BM25}}(d) - \mu_{\text{BM25}}}{\sigma_{\text{BM25}}}.$$

The combined score is:

$$s_{\text{CombSUM-z}}(d) = s'_{\text{dense}}(d) + s'_{\text{BM25}}(d).$$

Advantages.

- **Score alignment:** z-normalization puts both systems on common scale.
- **No manual α :** behaves like a balanced interpolation without tuning.
- **Complementarity:** rewards documents scoring well on either or both signals.

10.4 CombMNZ with z-score Normalization (CombMNZ-z)

Idea. CombMNZ amplifies the CombSUM signal by weighting documents by the number of contributing systems:

$$s_{\text{CombMNZ-z}}(d) = (s'_{\text{dense}}(d) + s'_{\text{BM25}}(d)) \cdot nz(d),$$

where $nz(d)$ is the number of non-zero (or non-negligible) contributions for document d .

Advantages.

- **Reward agreement:** boosts documents consistently supported by both BM25 and dense.
- **Noise suppression:** down-weights documents favored by only one system.
- **Often stronger hybrid:** CombMNZ variants often outperform simple sums.

10.5 Learned Linear Fusion (Least Squares, No α Constraint)

Idea. Instead of fixing how scores are combined we learn a linear fusion function. For each candidate document d , we use features:

$$X(d) = [s_{\text{dense}}(d), s_{\text{BM25}}(d), 1],$$

with binary target:

$$y(d) = \begin{cases} 1, & \text{if } d \text{ is relevant (from qrels),} \\ 0, & \text{otherwise.} \end{cases}$$

We fit a least-squares regression on a development set:

$$s_{\text{linear}}(d) = w_1 \cdot s_{\text{dense}}(d) + w_2 \cdot s_{\text{BM25}}(d) + b,$$

where w_1, w_2, b are learned to best separate relevant from non-relevant documents.

Advantages.

- **Data-driven weighting:** no heuristic design; the system learns importance of each signal.
- **No manual α :** the learned weights implicitly define the optimal balance.
- **Simple and efficient:** a lightweight learning-to-rank step requiring only linear regression on top-k candidates.

11 Cross-Attention Fusion Model: Theoretical Description

To move beyond classical score-level hybrid fusion, we implement a crossattention based representation fusion model that jointly encodes sparse BM25 signals and dense semantic vectors into a unified embedding space. The objective is to allow the model to learn interactions between lexical terms and dense semantic features, rather than merely combining independent retrieval scores.

11.1 Input Representation

Each document and query is decomposed into two complementary components:

- **Dense embedding:** A transformer-based embedding (e.g., MiniLM) providing global semantic information.
- **Sparse lexical representation:** A selected subset of the most informative BM25 terms, represented by their term IDs and BM25 weights, capturing exact match relevance and term frequency signals.

These two components are assembled into a short mixed token sequence:

$$[\text{DenseToken}, \text{BM25Term}_1, \text{BM25Term}_2, \dots, \text{BM25Term}_m],$$

where m is the number of selected lexical terms. The dense token embeds semantic information while the sparse tokens encode lexical importance.

11.2 Cross-Attention Fusion Mechanism

A lightweight Transformer Encoder processes the mixed sequence. Cross attention layers allow dense and sparse information to interact:

- The dense token attends to BM25 term tokens, enabling semantic features to incorporate term level lexical evidence
- BM25 term tokens attend back to the dense token, allowing lexical cues to be contextualized by semantic meaning.

Formally, let $X \in \mathbb{R}^{(m+1) \times d}$ be the input sequence matrix where the first row corresponds to the dense token. A Transformer Encoder layer computes:

$$X' = \text{TransformerEncoder}(X),$$

and the fused representation is taken as:

$$h = X'_0,$$

the output corresponding to the dense token index. This vector lies in a shared embedding space for queries and documents, allowing standard dotproduct retrieval.

Why We Use Very Few Training Epochs in the Cross-Attention Fusion Model?

The cross-attention fusion model combines dense vectors and sparse BM25 term signals through a small Transformer encoder. Although this is a powerful architecture, our dataset (TREC-COVID subset) is extremely small, containing only 17 queries, each with very few relevant documents. Because of this limited supervision, the model is highly prone to overfitting.

It happens when a neural model starts memorizing training triples instead of learning general fusion patterns. Since Transformers have high capacity, they can memorize all(query, positive,negative) triples after just a few passes. When this happens, the training loss keeps decreasing but evaluation metrics (nDCG@10, MRR@10, P@10, MAP@100) stop improving or even decline.

Dense embeddings already encode strong semantic information, while BM25 weights provide stable lexical signals. The fusion model only needs to fine-tune how these signals should interact and not learn complex semantics. Therefore, it converges extremely quickly and training beyond 1–3 epochs typically leads to overfitting, especially on such a small dataset.

So we deliberately keep epochs low:

1 epoch: learns basic interactions between dense and sparse signals

2–3 epochs: usually best performance

>3 epochs: model memorizes queries, metrics worsen

Thus, the recommended setting is 1–3 epochs, which provides sufficient learning without degrading generalization. This ensures the fused vectors remain strong and robust for scoring, retrieval, and evaluation.

11.3 Learning Objective (Optional)

To improve ranking quality, the model may be trained using a pairwise margin-based ranking loss. For a query q , relevant document d^+ , and non relevant document d^- , we minimise:

$$\mathcal{L}(q, d^+, d^-) = \max(0, \gamma - s(q, d^+) + s(q, d^-)),$$

where $s(q, d) = h(q)^\top h(d)$ is the similarity score and γ is a fixed margin.

Training encourages the model to learn which lexical cues and dense features most reliably correlate with relevance. Even without training, the model performs structural cross-attention fusion using randomly initialized parameters.

11.4 Inference and Retrieval

After fusion, both documents and queries are mapped into the shared fused embedding space:

$$q \mapsto h(q), \quad d \mapsto h(d).$$

Retrieval is performed using dot-product similarity:

$$\text{score}(q, d) = h(q)^\top h(d).$$

For evaluation we compute:

- **nDCG@10** for ranking quality
- **MRR@10** for early precision.

A TREC-formatted run file is generated to support external evaluation tools.

11.5 Summary

This cross-attention fusion model moves beyond traditional hybrid fusion approaches (e.g., weighted sums, RRF) by integrating:

- **dense semantic meaning** from transformer embeddings
- **term level lexical importance** from BM25 weights,

within a unified representational framework. By learning token-level interactions between sparse and dense signals, the model produces richer, task-specific embeddings for both documents and queries.

```

$ cd /mnt/c/Users/yourname/Desktop/yourname-hybrid-retrieval
$ python3 compare_systems.py
Using split: work/robotics/bel-trac-covid/subset_queries.json
System      nDCG@10  MRR@10  P@10  P@50  nDCG@100
BM25        0.8440  0.8121  0.7321  0.8148  0.8881
Dense       0.8728  0.8723  0.7284  0.8148  0.8814
Hybrid_Cross 0.8645  0.8680  0.8080  0.8148  0.8760
Hybrid_Sum  0.8621  0.8680  0.8020  0.8148  0.8862
Hybrid_Rerank 0.8721  0.8723  0.7284  0.8148  0.8814
Cross_Attn  0.8621  0.8621  0.8080  0.8148  0.8814
RRF         0.8721  0.8723  0.7284  0.8148  0.8814
CrossAtt_C  0.8620  0.8723  0.7221  0.8148  0.8800
Linear_L2   0.8620  0.8700  0.8148  0.8178  0.8115
BM25_HF    0.8611  0.8308  0.7088  0.8121  0.8511
DenseVec    0.8700  0.8723  0.7221  0.8147  0.8814
load metrics dir to work/robotics/bel-trac-covid/subset_queries.json

```

Figure 15: Evaluation

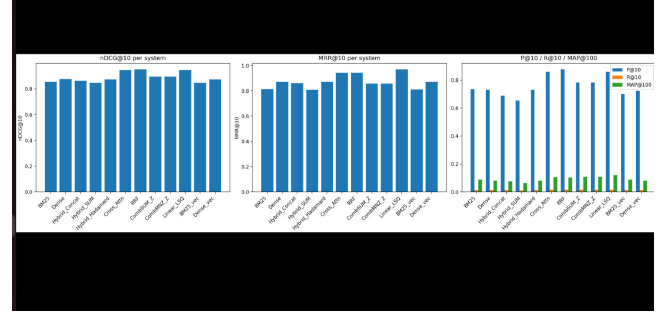


Figure 16: Graphs

12 Conclusion

We present a complete implementation and comparative analysis of hybrid sparse+dense retrieval across 23 BEIR datasets. The study confirms that linear score fusion when properly normalized is a strong and practical baseline. Our implementation offers a reproducible foundation for future hybrid retrieval research, including neural reranking, vector concatenation hybrids and hybrid ANN search structures.

Acknowledgments

We thank the faculty of IIT Bhilai for their support and guidance throughout this project.